SystemVerilog-$cast详解

$cast在子类与父类之间的复制

1 子类cast给父类：$cast(father_cls, child_cls)

之前有提到，$cast是将两个类型强制转换，$cast(A,B)，将B强制类型转换给A，应用在类上，就是句柄的强制赋值操作，那么在子类和父类之间的赋值呢？其实是原理是一样的，规则也只能是子类的句柄赋值给父类，如下图所示代码块：

```
4.              inner_cls i_a;

5.              function new();

6.                  i_a = new();

10.         class child_cls extends base_cls;

15.             base_cls base_0;

16.             child_cls child_0;

17.             child_0 = new();

18.             $cast(base_0, child_0);
```

且经过cast后，等同于赋值操作，即父类的句柄指向子类的对象，且只能访问子类对象的继承类的成员变量。仿一下看看：

```
program cls_cp;

    class inner_cls;
        int in_a = 200;
        function inner_cls copy();
            inner_cls a0;
            a0 = new();
            a0.in_a = this.in_a;
            return a0;
        endfunction
    endclass
    class base_cls;
       int a = 100;
        inner_cls i_a;
        function new();
            i_a = new();
        endfunction
        virtual function base_cls copy();
            base_cls b0;
            b0 = new();
            b0.a = this.a;
            b0.i_a = b0.i_a.copy();
            return b0;
        endfunction
        virtual function void display();
            $display("a in father cls is %d ", a);
        endfunction
    endclass

    class child_cls extends base_cls;
        int b = 300;
        function void display();
            $display("b in child cls is %d ", b);
        endfunction

    endclass

    initial begin
        base_cls cls_0;
        child_cls child_0;
        child_0 = new();
        //cls_0 = new();
        //child_0 = new cls_0;
        //cls_0 = child_0;
        $cast(cls_0, child_0);

        cls_0.display();
        child_0.display();
        $display("b in child_0 is %d", child_0.b);
        $display("a in child_0's father is %d", cls_0.a);
        $display("in_a in child_0's father is %d", cls_0.i_a.in_a);

        $display("a in father cls is %d", cls_0.a);
        $display("in_a in father cls is %d", cls_0.i_a.in_a);

        //$display("acc b in child_cls from cls_0 is %d", cls_0.b);

        child_0.a = 1000;
        child_0.i_a.in_a = 2000;

        $display("a in father cls is %d", cls_0.a);
        $display("in_a in father cls is %d", cls_0.i_a.in_a);
        //cls_0.a = 1000;
        //cls_0.i_a.in_a = 2000;
        //$display("b in child_0 is %d", child_0.b);
        //$display("a in child_0's father is %d", child_0.a);
        //$display("in_a in child_0's father is %d", child_0.i_a.in_a);

    end

endprogram
```

```
ICer@IC_EDA:~/my_blog
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
if [ -x ../simv ]; then chmod -x ../simv; fi
g++  -o ../simv    -Wl,-rpath-link=./ -Wl,-rpath='$ORIGIN'/simv.daidir/ -Wl,
l,-rpath='$ORIGIN'/simv.daidir//scsim.db.dir  -rdynamic  -Wl,-rpath=/home/sy
1/linux64/lib -L/home/synopsys/vcs-mx/O-2018.09-1/linux64/lib     _29310_arc
1.so _csrc0.so  SIM_l.o  _csrc0.so     rmapats_mop.o rmapats.o rmar.o rmar_r
r_llvm_0_0.o          -lzerosoft_rt_stubs -lvirsim -lerrorinf -lsnpsmalloc -
ofile -luclinative /home/synopsys/vcs-mx/O-2018.09-1/linux64/lib/vcs_tls.o
sucli -Wl,-no-whole-archive          /home/synopsys/vcs-mx/O-2018.09-1/linux
new.o -ldl  -lc -lm -lpthread -ldl
../simv up to date
Chronologic VCS simulator copyright 1991-2018
Contains Synopsys proprietary information.
Compiler version O-2018.09-1_Full64; Runtime version O-2018.09-1_Full64;  Ma
b in child cls is         300
b in child cls is         300
b in child_0 is         300
a in child_0's father is         100
in_a in child_0's father is         200
a in father cls is         100
in_a in father cls is         200
a in father cls is         1000
in_a in father cls is         2000
$finish at simulation time                   0
           V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.290 seconds;       Data structure size:   0.0Mb
Sun May  7 00:14:18 2023
CPU time: .251 seconds to compile + .222 seconds to elab + .252 seconds to l
mulation
[ICer@IC_EDA /home/ICer/my_blog]$
```

具备多态特性

指向同一块空间

CSDN @Lucky是一名ICer

反过来cast，直接报错：

```
1  program cls_cp;
2
3      class inner_cls;
4          int in_a = 200;
5          function inner_cls copy();
6              inner_cls a0;
7              a0 = new();
8              a0.in_a = this.in_a;
9              return a0;
10         endfunction
11     endclass
12     class base_cls;
13        int a = 100;
14         inner_cls i_a;
15         function new();
16             i_a = new();
17         endfunction
18         virtual function base_cls copy();
19             base_cls b0;
20             b0 = new();
21             b0.a = this.a;
22             b0.i_a = b0.i_a.copy();
23             return b0;
24         endfunction
25         virtual function void display();
26             $display("a in father cls is %d ", a);
27         endfunction
28     endclass
29
30     class child_cls extends base_cls;
31         int b = 300;
32         function void display();
33             $display("b in child cls is %d ", b);
34         endfunction
35
36     endclass
37
38     initial begin
39         base_cls cls_0;
40         child_cls child_0;
41         child_0 = new();
42         //cls_0 = new();
43         //child_0 = new cls_0;
44         //cls_0 = child_0;
45         //$cast(cls_0, child_0);
46         $cast(child_0, cls_0);
47
48         cls_0.display();
49         child_0.display();
50         $display("b in child_0 is %d", child_0.b);
51         $display("a in child_0's father is %d", cls_0.a);
52         $display("in_a in child_0's father is %d", cls_0.i_a.in_a);
53
54         $display("a in father cls is %d", cls_0.a);
55         $display("in_a in father cls is %d", cls_0.i_a.in_a);
56
57         //$display("acc b in child_cls from cls_0 is %d", cls_0.b);
58
59         child_0.a = 1000;
60         child_0.i_a.in_a = 2000;
61
62         $display("a in father cls is %d", cls_0.a);
63         $display("in_a in father cls is %d", cls_0.i_a.in_a);
64         //cls_0.a = 1000;
65         //cls_0.i_a.in_a = 2000;
66         //$display("b in child_0 is %d", child_0.b);
67         //$display("a in child_0's father is %d", child_0.a);
68         //$display("in_a in child_0's father is %d", child_0.i_a.in_a);
69
70     end
71
72 endprogram
```

```
ICer@IC_EDA:~/my_blog
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
l,-rpath='$ORIGIN'/simv.daidir//scsim.db.dir  -rdynamic  -Wl,-rpath=/home/synopsys
1/linux64/lib -L/home/synopsys/vcs-mx/O-2018.09-1/linux64/lib     _30252_archive_1
1.so _csrc0.so  SIM_l.o  _csrc0.so     rmapats_mop.o rmapats.o rmar.o rmar_nd.o
r_llvm_0_0.o          -lzerosoft_rt_stubs -lvirsim -lerrorinf -lsnpsmalloc -lvfs
ofile -luclinative /home/synopsys/vcs-mx/O-2018.09-1/linux64/lib/vcs_tls.o   -Wl,-
sucli -Wl,-no-whole-archive          /home/synopsys/vcs-mx/O-2018.09-1/linux64/lib
new.o -ldl  -lc -lm -lpthread -ldl
../simv up to date
Chronologic VCS simulator copyright 1991-2018
Contains Synopsys proprietary information.
Compiler version O-2018.09-1_Full64;  Runtime version O-2018.09-1_Full64;   May  7 (

Error-[NOA] Null object access
cls_cp.sv,  48
  The object at dereference depth 0 is being used before it was
  constructed/allocated.
  Please make sure that the object is allocated before using it.

  #0 in unnamed$$_0 at cls_cp.sv:48
  #1 in cls_cp


          V C S   S i m u l a t i o n   R e p o r t
Time:  0
CPU Time:       0.280 seconds;       Data structure size:    0.0Mb
Sun May  7 00:16:38 2023
Note:  Execution of simv exited with code 1
CPU time:  .231 seconds to compile + .179 seconds to elab + .244 seconds to link +
mulation
[ICer@IC_EDA /home/ICer/my_blog]$
```

CSDN @Lucky是一名ICer

2. 父类cast给子类： $cast(child_cls, father_cls)

上文提到，父类是不能cast给子类的，那么在什么情况下可以呢？其实很容易想到，就是在父类的句柄指向子类的

对象的时候，cast可以生效，如下图所示代码块：

```
4.              inner_cls i_a;
5.              function new();
6.                  i_a = new();
10.         class child_cls extends base_cls;
15.             base_cls base_0;
16.             child_cls child_0;
17.             child_0 = new();
18.             base_0 = child_0;
19.             $cast(child_0, base_0);
```

这样的化，通过父类的cast，其实就是子类的对象，同样的父类也不能访问子类本身的成员变量，父类改变自身的成员变量，子类调用自己父类的成员变量时也会发生改变。仿一下看看：

```
1  program cls_cp;
2
3      class inner_cls;
4          int in_a = 200;
5          function inner_cls copy();
6              inner_cls a0;
7              a0 = new();
8              a0.in_a = this.in_a;
9              return a0;
10         endfunction
11     endclass
12     class base_cls;
13        int a = 100;
14         inner_cls i_a;
15         function new();
16             i_a = new();
17         endfunction
18         virtual function base_cls copy();
19             base_cls b0;
20             b0 = new();
21             b0.a = this.a;
22             b0.i_a = b0.i_a.copy();
23             return b0;
24         endfunction
25         virtual function void display();
26             $display("a in father cls is %d ", a);
27         endfunction
28     endclass
29
30     class child_cls extends base_cls;
31         int b = 300;
32         function void display();
33             $display("b in child cls is %d ", b);
34         endfunction
35
36     endclass
37
38     initial begin
39         base_cls cls_0;
40         child_cls child_0;
41         //child_cls child_1;
42         child_0 = new();
43         //cls_0 = new();
44         //child_0 = new cls_0;
45         cls_0 = child_0;
46         //$cast(cls_0, child_0);
47         $cast(child_0, cls_0);
48
49         cls_0.display();
50         child_0.display();
51         $display("b in child_0 is %d", child_0.b);
52         $display("a in child_0's father is %d", cls_0.a);
53         $display("in_a in child_0's father is %d", cls_0.i_a.in_a);
54
55         $display("a in father cls is %d", cls_0.a);
56         $display("in_a in father cls is %d", cls_0.i_a.in_a);
57
58         //$display("acc b in child_cls from cls_0 is %d", cls_0.b);
59
60         child_0.a = 1000;
61         child_0.i_a.in_a = 2000;
62
63         $display("a in father cls is %d", cls_0.a);
64         $display("in_a in father cls is %d", cls_0.i_a.in_a);
65         //cls_0.a = 1000;
66         //cls_0.i_a.in_a = 2000;
67         //$display("b in child_0 is %d", child_0.b);
68         //$display("a in child_0's father is %d", child_0.a);
69         //$display("in_a in child_0's father is %d", child_0.i_a.in_a);
70
71     end
72
73 endprogram
```

```
ICer@IC_EDA:~/my_blog
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
if [ -x ../simv ]; then chmod -x ../simv; fi
g++  -o ../simv     -Wl,-rpath-link=./ -Wl,-rpath='$ORIGIN'/simv.daidir/ -Wl,-
l,-rpath='$ORIGIN'/simv.daidir//scsim.db.dir  -rdynamic  -Wl,-rpath=/home/syn
1/linux64/lib -L/home/synopsys/vcs-mx/O-2018.09-1/linux64/lib      _34012_arch
1.so _csrc0.so  SIM_l.o  _csrc0.so      rmapats_mop.o rmapats.o rmar.o rmar_nd
r_llvm_0_0.o           -lzerosoft_rt_stubs -lvirsim -lerrorinf -lsnpsmalloc -l
ofile -luclinative /home/synopsys/vcs-mx/O-2018.09-1/linux64/lib/vcs_tls.o
sucli -Wl,-no-whole-archive          /home/synopsys/vcs-mx/O-2018.09-1/linux6
new.o -ldl  -lc -lm -lpthread -ldl
../simv up to date
Chronologic VCS simulator copyright 1991-2018
Contains Synopsys proprietary information.
Compiler version O-2018.09-1_Full64;  Runtime version O-2018.09-1_Full64;  May
b in child cls is         300
b in child cls is         300
b in child_0 is         300
a in child_0's father is         100
in_a in child_0's father is         200
a in father cls is         100
in_a in father cls is         200
a in father cls is         1000
in_a in father cls is         2000
$finish at simulation time                   0
           V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.250 seconds;       Data structure size:   0.0Mb
Sun May  7 00:26:32 2023
CPU time: .244 seconds to compile + .203 seconds to elab + .213 seconds to li
mulation
[ICer@IC_EDA /home/ICer/my_blog]$
```

多态特性

改一个，另一个也改变

CSDN @Lucky是一名ICer

修改父类的成员变量，子类也会跟着改：

```
program cls_cp;

    class inner_cls;
        int in_a = 200;
        function inner_cls copy();
            inner_cls a0;
            a0 = new();
            a0.in_a = this.in_a;
            return a0;
        endfunction
    endclass
    class base_cls;
       int a = 100;
        inner_cls i_a;
        function new();
            i_a = new();
        endfunction
        virtual function base_cls copy();
            base_cls b0;
            b0 = new();
            b0.a = this.a;
            b0.i_a = b0.i_a.copy();
            return b0;
        endfunction
        virtual function void display();
            $display("a in father cls is %d ", a);
        endfunction
    endclass

    class child_cls extends base_cls;
        int b = 300;
        function void display();
            $display("b in child cls is %d ", b);
        endfunction

    endclass

    initial begin
        base_cls cls_0;
        child_cls child_0;
        //child_cls child_1;
        child_0 = new();
        //cls_0 = new();
        //child_0 = new cls_0;
        cls_0 = child_0;
        //$cast(cls_0, child_0);
        $cast(child_0, cls_0);

        cls_0.display();
        child_0.display();
        $display("b in child_0 is %d", child_0.b);
        $display("a in child_0's father is %d", cls_0.a);
        $display("in_a in child_0's father is %d", cls_0.i_a.in_a);

        $display("a in father cls is %d", cls_0.a);
        $display("in_a in father cls is %d", cls_0.i_a.in_a);

        //$display("acc b in child_cls from cls_0 is %d", cls_0.b);

        //child_0.a = 1000;
        //child_0.i_a.in_a = 2000;

        //$display("a in father cls is %d", cls_0.a);
        //$display("in_a in father cls is %d", cls_0.i_a.in_a);
        cls_0.a = 1000;
        cls_0.i_a.in_a = 2000;
        $display("b in child_0 is %d", child_0.b);
        $display("a in child_0's father is %d", child_0.a);
        $display("in_a in child_0's father is %d", child_0.i_a.in_a);

    end

endprogram
```

```
ICer@IC_EDA:~/my_blog
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
g++  -o ../simv    -Wl,-rpath-link=./ -Wl,-rpath='$ORIGIN'/simv.daidir/ -Wl,-
l,-rpath='$ORIGIN'/simv.daidir//scsim.db.dir  -rdynamic  -Wl,-rpath=/home/syn
1/linux64/lib -L/home/synopsys/vcs-mx/O-2018.09-1/linux64/lib     _33577_arch
1.so _csrc0.so  SIM_l.o  _csrc0.so     rmapats_mop.o rmapats.o rmar.o rmar_nd
r_llvm_O_0.o          -lzerosoft_rt_stubs -lvirsim -lerrorinf -lsnpsmalloc -l
ofile -luclinative /home/synopsys/vcs-mx/O-2018.09-1/linux64/lib/vcs_tls.o
sucli -Wl,-no-whole-archive          /home/synopsys/vcs-mx/O-2018.09-1/linux6
new.o -ldl  -lc -lm -lpthread -ldl
../simv up to date
Chronologic VCS simulator copyright 1991-2018
Contains Synopsys proprietary information.
Compiler version O-2018.09-1_Full64; Runtime version O-2018.09-1_Full64;  May
b in child cls is          300
b in child cls is          300
b in child_0 is          300
a in child_0's father is          100
in_a in child_0's father is          200
a in father cls is          100
in_a in father cls is          200
b in child_0 is          300
a in child_0's father is         1000
in_a in child_0's father is         2000
$finish at simulation time                   0
           V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.230 seconds;       Data structure size:   0.0Mb
Sun May  7 00:25:52 2023
CPU time: .241 seconds to compile + .169 seconds to elab + .212 seconds to li
mulation
[ICer@IC_EDA /home/ICer/my_blog]$
```

CSDN @Lucky是一名ICer

实际上，这种赋值是没有意义的，类的赋值要体现在不同的对象之间，用这种方式可以实现父类作为桥梁赋值给另一个子类的对象，在UVM中得到了大量的使用。

```
program cls_cp;

    class inner_cls;
        int in_a = 200;
        function inner_cls copy();
            inner_cls a0;
            a0 = new();
            a0.in_a = this.in_a;
            return a0;
        endfunction
    endclass
    class base_cls;
       int a = 100;
        inner_cls i_a;
        function new();
            i_a = new();
        endfunction
        virtual function base_cls copy();
            base_cls b0;
            b0 = new();
            b0.a = this.a;
            b0.i_a = b0.i_a.copy();
            return b0;
        endfunction
        virtual function void display();
            $display("a in father cls is %d ", a);
        endfunction
    endclass

    class child_cls extends base_cls;
        int b = 300;
        function void display();
            $display("b in child cls is %d ", b);
        endfunction

    endclass

    initial begin
        base_cls cls_0;
        child_cls child_0;
        child_cls child_1;
        child_0 = new();
        //cls_0 = new();
        //child_0 = new cls_0;
        cls_0 = child_0;
        //$cast(cls_0, child_0);
        $cast(child_1, cls_0);

        cls_0.display();
        child_0.display();
        child_1.display();
        $display("b in child_0 is %d", child_0.b);
        $display("a in child_0's father is %d", child_0.a);
        $display("in_a in child_0's father is %d", child_0.i_a.in_a);

        $display("b in child_1 is %d", child_1.b);
        $display("a in child_1's father is %d", child_1.a);
        $display("in_a in child_1's father is %d", child_1.i_a.in_a);

        $display("a in father cls is %d", cls_0.a);
        $display("in_a in father cls is %d", cls_0.i_a.in_a);

        //$display("acc b in child_cls from cls_0 is %d", cls_0.b);

        child_0.b = 5000;
        child_0.a = 1000;
        child_0.i_a.in_a = 2000;

        $display("========================");
        $display("a in father cls is %d", cls_0.a);
        $display("in_a in father cls is %d", cls_0.i_a.in_a);

        $display("b in child_1 is %d", child_1.b);
        $display("a in child_1's father is %d", child_1.a);
        $display("in_a in child_1's father is %d", child_1.i_a.in_a);

        //cls_0.a = 1000;
        //cls_0.i_a.in_a = 2000;
        //$display("b in child_0 is %d", child_0.b);
        //$display("a in child_0's father is %d", child_0.a);
        //$display("in_a in child_0's father is %d", child_0.i_a.in_a);

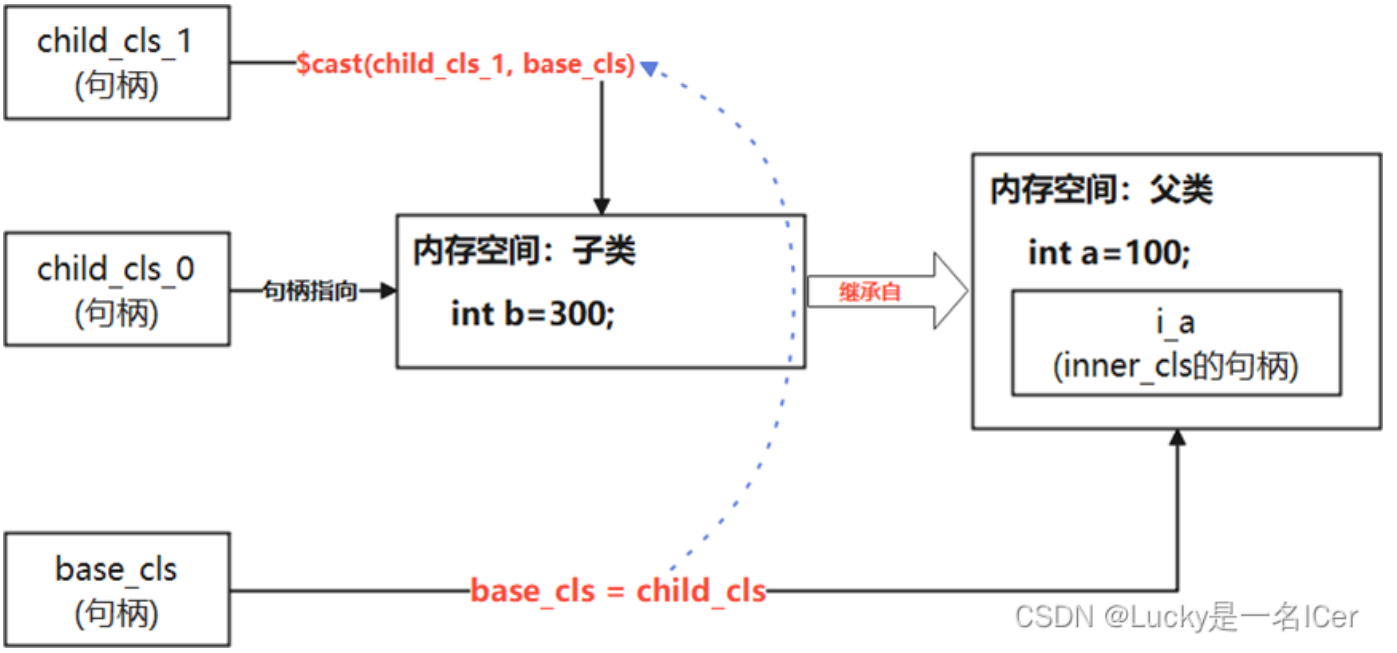
    end

endprogram
```

```
ICer@IC_EDA:~/my_blog
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
new.o -ldl  -lc -lm -lpthread -ldl
../simv up to date
Chronologic VCS simulator copyright 1991-2018
Contains Synopsys proprietary information.
Compiler version O-2018.09-1_Full64;  Runtime version O-2018.09-1_Full64;  May
b in child cls is          300
b in child cls is          300
b in child cls is          300
b in child_0 is          300
a in child_0's father is          100
in_a in child_0's father is          200
b in child_1 is          300
a in child_1's father is          100
in_a in child_1's father is          200
a in father cls is          100
in_a in father cls is          200
========================
a in father cls is         1000
in_a in father cls is         2000
b in child_1 is         5000
a in child_1's father is         1000
in_a in child_1's father is         2000
$finish at simulation time                   0
           V C S   S i m u l a t i o n   R e p o r t
Time: 0
CPU Time:      0.240 seconds;       Data structure size:   0.0Mb
Sun May  7 00:30:17 2023
CPU time: .262 seconds to compile + .185 seconds to elab + .262 seconds to li
mulation
[ICer@IC_EDA /home/ICer/my_blog]$
```

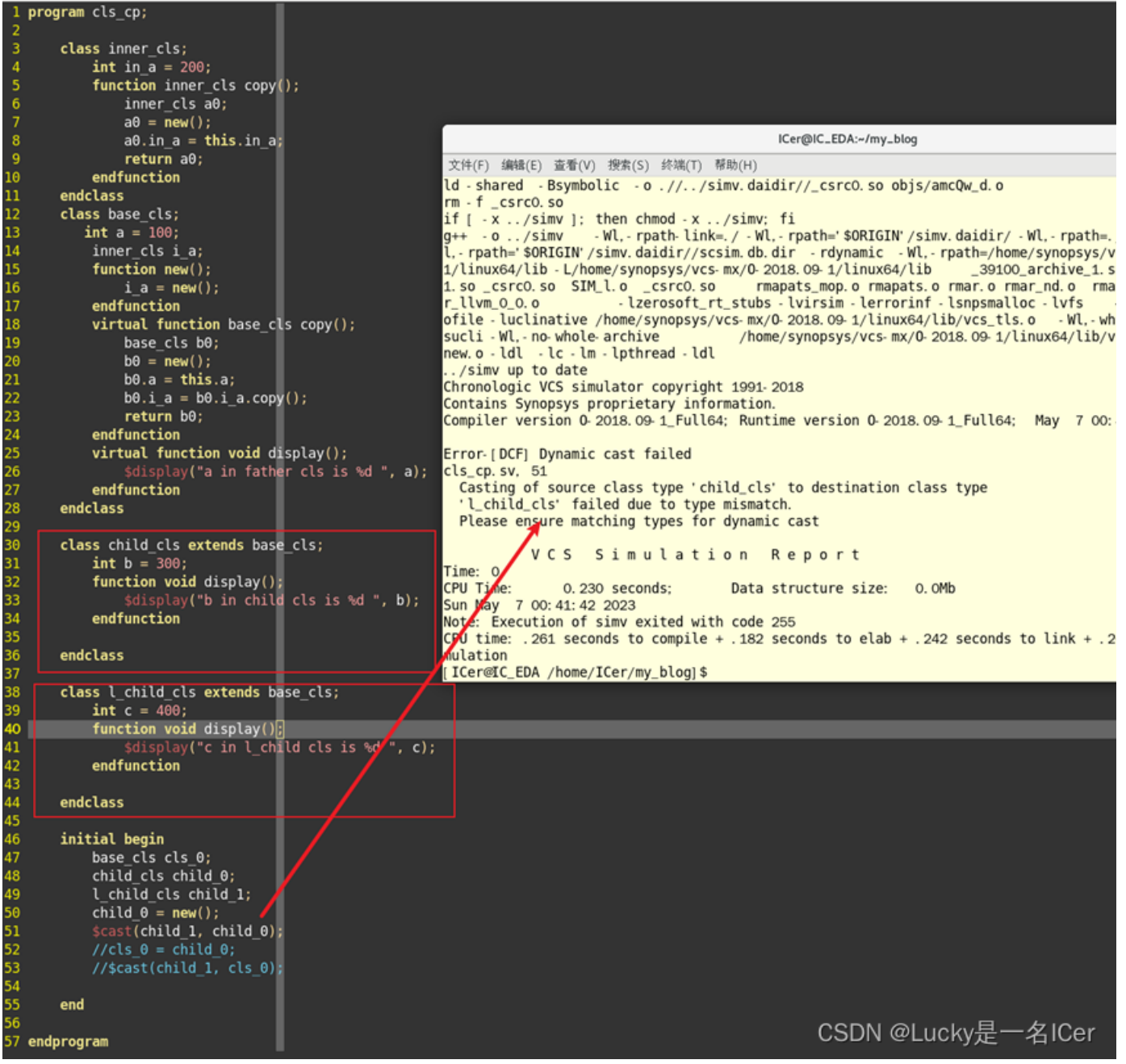
CSDN @Lucky是一名ICer

改一个，另一个也跟着改变，读者可以自行尝试改变child_1的成员变量，以及改变cls_0的成员变量，看是否另外的两个类成员变量也发生改变。值得注意的是，cls_0依然无法访问child_0和child_1中的成员变量。过程如下图所示：

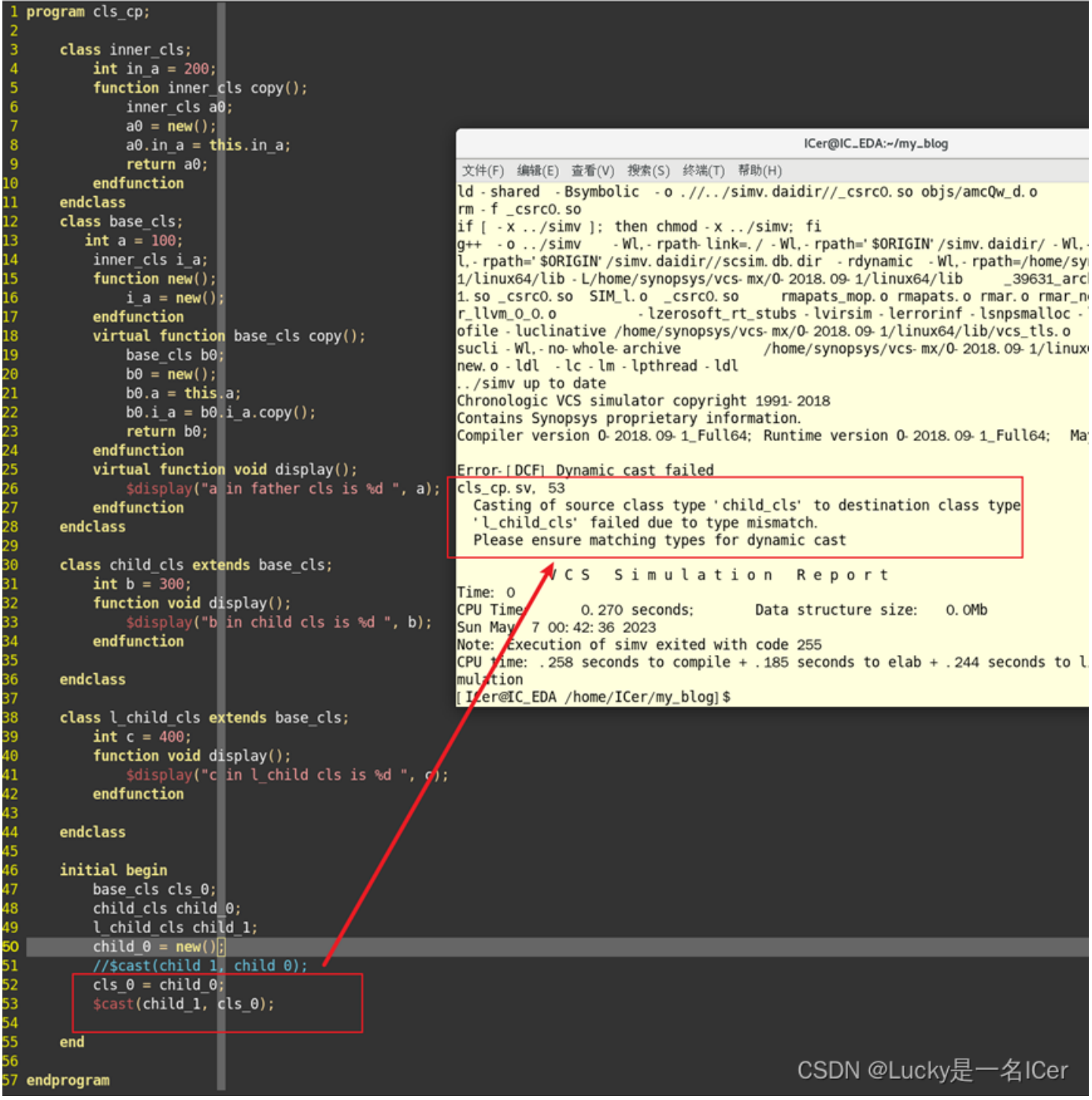


3. 两个不同的子类之间的cast

同一个子类的不同对象之间是可以cast的，那么不同子类直接能否cast？能否通过父类进行cast？这里先给出结论：**都是不可以的，原因是cast在对于类进行操作时，认为相同类型的两个句柄，可以cast，那么如果两个不同子类声明的句柄，cast会认为是不同的类型，因此不能转换，即使他们继承自一个父类，同样通过父类的cast也是不可以的。**如下图仿真：



直接报错，通过父类进行不同子类之间的cast，也不可以：



$cast在子类与父类之间深拷贝的用法

在3.2节理解了cast在父类和子类的用法后，本节以一个具体的实例，说明cast在深拷贝中的应用，例如如下代码块：

```
4.                int in_a = 200;

5.                function void copy(inner_cls ii);
```

```
6.                  ii.in_a = this.in_a;

11.             inner_cls i_a;

12.             function new();

13.                 i_a = new();

15.             virtual function void copy(base_cls i_c);

16.                 i_c.a = this.a;

17.                 i_c.i_a.copy(this.i_a);

19.             virtual function void display();

20.                 $display("a in father cls is %d ", a);

24.         class child_cls extends base_cls;

26.             function void display();

27.                 $display("b in child cls is %d ", b);

30.             function void copy(base_cls c);

31.                 child_cls cc;

32.                 super.copy(c);

33.                 $cast(cc, c);

34.                 cc.b = this.b;

40.             child_cls child_0;

41.             child_cls child_1;

42.             child_0 = new();
```

```
43.             child_1 = new();

44.             child_1.copy(child_0);
```

分析：**在子类中重载了父类的copy函数，重载的过程是，先声明了一个子类，然后调用父类的copy函数，将父类的成员变量进行深拷贝赋值，包括父类的深层次成员变量inner_cls，然后再通过$cast方法，将父类赋值给子类，这样在子类cc中，就有了父类成员变量的复制本，最后将子类的成员变量进行复制，完成深拷贝。**

此处其实有一个问题，就是之前不是说父类不能够cast给子类吗？为什么这里可以写，因为在实际调用的时候，是两个子类之间的复制，通过调用子类的copy函数，copy函数的参数传递的是子类的句柄，在函数传参的过程中，也就将形式参数父类的句柄指向了实参子类的对象，因此可以cast。

仿真结果：



读者可以自行尝试注释行的仿真，结果应该是一样的。

UVM里用到了大量这样的方法，掌握这个原理，对于理解UVM的应用，以及SystemVerilog更高级的建模，都十分有帮助。